Confusão e Difusão em Imagens

Programação de Baixo Nível – T1

Grupo: Martim Raymann, Felipe Araújo

Link para o git: https://github.com/MartimRaymann/bagunceitor

1. Objetivo .

Implementar em C uma transformação reversível de uma imagem RGB. A confusão modifica os valores dos componentes de cor e a difusão reorganiza as posições dos pixels. A recuperação deve reproduzir exatamente o buffer RGB original, e não apenas uma imagem visualmente semelhante.

A solução parte do código base fornecido e utiliza **stb_image** para leitura e **stb_image_write** para gravação. As regras que foram adotadas são soma circular por canal e deslocamentos circulares por linha e coluna.

2. Representação e sequência.

```
typedef struct {unsigned char r, g, b;} Pixel;
Original → confundir → difundir → Bagunçada
Bagunçada → desdifundir → desconfundir → Recuperada
```

Cada pixel é formado por três componentes de um byte, de 0 a 255. A estrutura Img guarda largura, altura, canais e um ponteiro para uma região contínua de pixels. O carregamento solicita três canais; entradas RGBA têm a transparência descartada. A garantia de recuperação se refere aos componentes RGB decodificados, não aos bytes do arquivo JPEG de entrada.

3. Relação com o enunciado.

Exigência	Implementação
Substituir valores e possuir inversa	confundir () e desconfundir ()
Permutar posições e possuir inversa	difundir () e desdifundir ()
Ponteiros e aritmética de memória	Pixel*, índices lineares e malloc/free
Recuperação integral	Comparação de todos os componentes RGB

4. Confusão e sua inversa.

Com chave K, a transformação soma um deslocamento diferente a cada canal. O módulo 256 faz os valores circularem dentro do intervalo de um byte. Na execução demonstrada, K = 42.

```
R' = (R + K) mod 256
G' = (G + 2K) mod 256
B' = (B + 3K) mod 256
```

Para um canal de valor v e deslocamento d normalizado entre 0 e 255, a operação é C(v) = (v + d) mod 256 e sua inversa é $C^{-1}(v') = (v' + 256 - d) \bmod 256$. A soma seguida da subtração do mesmo deslocamento retorna v. O +256 evita resto negativo em C; os deslocamentos de G e B são reduzidos módulo 256 antes da subtração.

```
saída->r = (entrada->r + chave) % 256;
saída->g = (entrada->g + 2 * chave) % 256;
saída->b = (entrada->b + 3 * chave) % 256;
```

A confusão percorre os pixels linearmente, por ponteiros, sem alterar suas posições. A leitura de entrada e a escrita de saída são feitas nos campos r, g e b de Pixel.

5. Difusão e sua inversa.

Se W é a largura e H a altura, usamos as regras abaixo. As coordenadas começam em zero. O pixel RGB inteiro é copiado, mantendo seus três componentes juntos.

```
Deslocamento da linha y: s(y) = (K + y²) mod W
Nova coluna: x’ = (x + s(y)) mod W
Deslocamento da coluna x: t(x) = (K + x²) mod H
Nova linha: y’ = (y + t(x)) mod H
```

As linhas são deslocadas para a direita, gravando no buffer temporário, depois as colunas desse resultado são deslocadas para baixo, gravando no destino. Os pixels que ultrapassam uma borda reaparecem na borda oposta.

Todos os pixels da linha ou coluna continuam presentes uma única vez. A composição das duas etapas também é reversível. Na recuperação, desfazemos primeiro as colunas e depois as linhas, buscando os pixels nas posições às quais foram enviados:

```
Desfazer colunas: temp[y][x] = entrada [(y + t(x)) mod H] [x]
Desfazer linhas: saída[y][x] = temp[y] [(x + s(y)) mod W]
```

As duas inversas são funções separadas. No ciclo completo, a inversa da difusão é aplicada antes da inversa da confusão, conforme a ordem exigida nos slides.

6. Manipulação de memória.

O programa mantém três buffers: original, trabalho e temporária. A original fica intacta. Os dois auxiliares são alocados com malloc e reutilizados durante as etapas.

```
size_t total = (size_t)original.width * original.height;
size_t bytes = total * sizeof(Pixel);
trabalho.pixels = malloc(bytes);
temporaria.pixels = malloc(bytes);
```

Atribuir uma Img a outra copia os metadados e o ponteiro, não os pixels. Por isso os ponteiros dos auxiliares são substituídos por novas alocações. A aritmética pixels + y * largura + x encontra o endereço de um pixel; o operador * acessa o conteúdo desse endereço.

Na difusão, origem e destino podem ser o mesmo buffer porque a primeira etapa termina de copiar toda a origem para o temporário antes de a segunda escrever no destino. O temporário deve ser distinto. Após desfazer as posições, desconfundir escreve a recuperação final nesse temporário.

São verificadas falhas de leitura, alocação, gravação e dimensões suportadas. O buffer carregado é liberado com **stbi_image_free**; os auxiliares, com free. Para a imagem demonstrada, os três buffers RGB ocupam 11.059.200 bytes, sem contar estruturas e memória interna das bibliotecas.

7. Execução e arquivos gerados.

No terminal:

```
mingw32-make
./bagunceitor.exe predio32.jpg
```

Arquivo	Conteúdo
original.png	RGB carregado antes das transformações
baguncada.png	Confusão seguida de difusão
recuperada.png	Resultado das duas inversas

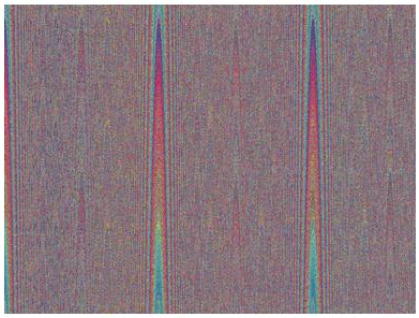
8. Resultados Obtidos

Execução com predio32.jpg, chave 42, resolução 1280 × 960. O terminal apresenta:

```
Load: 1280 x 960 | entrada: 3 canais | memoria: RGB
Processando com chave 42...
Pixels comparados: 1228800
Componentes RGB diferentes: 0
SUCESSO: recuperacao exata e PNGs salvos.
```



Original



Bagunçada



Recuperada

Verificação	Resultado

Confusão e inversa: 256 valores × 256 chaves	Passou nos três canais e nas posições testadas
Difusão: 1×1, 1×7, 7×1, 2×3, 3×2, 5×7 e 8×8	Passou; sem perda ou duplicação
Ciclo completo e comparação com diferenças inseridas	Passou; comparação detectou as alterações
Carregamento de imagens cinza e RGBA como RGB	Passou
PNGs salvos e recarregados	Recuperada igual; bagunçada diferente

9. Conclusão e uso de IA.

A execução comparou 1.228.800 pixels e encontrou zero componentes RGB diferentes. As operações isoladas e a composição passaram nos testes executados. A soma circular e as rotações preservam informação e possuem inversas explícitas; isso não implica segurança criptográfica.

Foi utilizado a IA como apoio à compreensão do enunciado, proposta das regras, testes e auxílio no código.